

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Факультет Компьютерных наук

Кафедра программирования и информационных технологий

Мобильное приложение ZadachOk для распределения семейных обязанностей

Курсовая работа

Направление: 09.03.04. Программная инженерия

Зав. Кафедрой _____ д. ф.-м. н, доцент С.Д. Махортов
Руководитель _____ ст. преподаватель В.С. Тарасов
Руководитель практики _____ Е.Д. Проскуряков
Обучающийся _____ В.С. Мосякин, 3 курс, д/о
Обучающийся _____ И.В. Четвергов, 3 курс, д/о
Обучающийся _____ К.А. Лышова, 3 курс, д/о
Обучающийся _____ А.В. Зыков, 3 курс, д/о
Обучающийся _____ А.В. Беляев, 3 курс, д/о
Обучающийся _____ Д.В. Круглов, 3 курс, д/о

Воронеж 2025

СОДЕРЖАНИЕ

Определения, обозначения и сокращения	4
Введение	5
1 Постановка задачи	6
1.1 Цели создания системы.....	6
1.2 Функциональные требования к разрабатываемой системе	7
1.3 Нефункциональные требования к разрабатываемой системе	7
2 Анализ предметной области	10
2.1 Конкурентный анализ	10
2.2 Определение ЦА и потребностей.....	12
3 Реализация	13
3.1 Frontend.....	13
3.1.1 Локальное состояние (виджетный уровень)	13
3.1.2 Глобальное состояние (прикладной уровень)	13
3.1.3 Персистентное состояние (хранение данных)	15
3.1.4 Структура проекта.....	15
3.2 Backend	18
3.2.1 Общая архитектура	18
3.2.2 Контроллеры	19
3.2.3 Сервисный уровень	19
3.2.4 Репозитории и доступ к данным	19
3.2.5 Обработка ошибок и безопасность	20
3.3 БД и СУБД	20
3.3.1 Описание базы данных AC ZadachOk	20
3.3.2 Безопасность, производительность и резервирование.....	22
3.4 Интеграция искусственного интеллекта (переименовать)	23
3.5 Развёртывание и запуск (переименовать)	23
3.5.1 Процесс развертывания	23
3.5.2 Финальная архитектура и реализация	24
3.6 CI/CD и тестирование.....	25
3.7 Функциональные возможности	26
3.7.1 Роли пользователей в системе.....	26
3.7.2 Функциональные возможности.....	27

(здесь описать каждый пользовательский путь)	27
3.7.3 Режимы функционирования системы.....	27
Заключение.....	29
Список использованных источников	31

Определения, обозначения и сокращения

Термин	Определение

Введение

В современном мире самоорганизация и распределение обязанностей в семье становятся всё более актуальными. Многие люди сталкиваются с проблемой забывчивости, отсутствия мотивации и неэффективного планирования совместных задач. Существующие таск—трекеры часто не учитывают семейный контекст и не предоставляют инструментов для мотивации участников.

Приложение ЗадачОк решает эти проблемы, предлагая:

- Распределение задач между членами семьи;
- Геймификацию с системой вознаграждений;
- Умные уведомления на основе ИИ для повышения вовлечённости;

Система предназначена для совместного ведения списка задач с возможностью создания групп пользователей. В зависимости от установленных администратором условий, участники могут получать вознаграждение в виде внутренней некоммерческой валюты. В приложении предусмотрен встроенный магазин, в котором данная валюта может быть использована для приобретения доступных товаров или услуг.

Проект актуален, так как люди часто игнорируют стандартные напоминания. ИИ генерирует персонализированные и шуточные сообщения, повышая внимание пользователей. Также к этому можно отнести систему баллов и виртуального магазина, которая стимулирует пользователя нашего приложения к выполнению задач. К тому же стоит отметить, что большинство таск—трекеров не адаптированы для семейного использования, что ставит наше приложение в более выигрышную позицию на рынке нашей ЦА.

1 Постановка задачи

1.1 Цели создания системы

- В течение 6 месяцев после запуска приложения планируется обеспечить привлечение 100 активных пользователей, под которыми понимаются регулярно взаимодействующие с системой лица, выполняющие ключевые действия: создание и завершение задач, использование виртуального магазина и участие в системе вознаграждений;
- Достичь средней оценки удовлетворённости интеграцией ИИ и геймификации не менее 7 баллов из 10 (где 1 — крайне неудовлетворительно, 10 — исключительно высокий уровень удовлетворённости) на основе анализа метрик приложения, включая частоту взаимодействий и конверсию покупки товаров в виртуальном магазине;
- В течение 6 месяцев после запуска приложения планируется достичь конверсии 5% пользователей в платящих клиентов за счет внедрения премиум—подписки (см. ПРИЛОЖЕНИЕ ...) с расширенным функционалом и рекламной модели с партнерской интеграцией.

1.2 Функциональные требования к разрабатываемой системе

- Создание и управление задачами;
- Система баллов и управление виртуальным магазином;
- Генерация текста для push—уведомлений с использованием с использованием ИИ;
- Поддержка офлайн—режима;
- Возможность редактировать личный профиль и просматривать личную статистику.

1.3 Нефункциональные требования к разрабатываемой системе

Производительность:

- Время загрузки календаря и списка задач не должно превышать 30 секунд при стабильном интернет—соединении со скоростью не менее 5 Мбит/с;
- Синхронизация данных между устройствами должна завершаться в течение 60 секунд после восстановления сетевого соединения;
- На устройстве, соответствующем минимальным системным требованиям, операции в оффлайн—режиме должны выполняться без ощутимых задержек;
- Формирование личной статистики по выполненным задачам должно занимать не более 15 секунд;
- Отображение содержимого виртуального магазина должно происходить в течение 40 секунд после открытия раздела.

Надежность:

- Система должна обеспечивать бесперебойную работу 70% времени в течение суток;

- Автоматическое сохранение данных должно происходить каждые 5 минут при активной сессии;
- В случае сбоя должно сохраняться не менее 70% пользовательских данных;
- Логирование всех критических ошибок **на сервере** с фиксацией времени, типа ошибки и контекста возникновения;

Безопасность:

- Все передаваемые данные должны шифроваться по протоколу HTTPS;
- Пароли пользователей должны храниться в хешированном виде;
- Сессионные токены должны иметь срок действия не более 24 часов;

Совместимость:

- Поддержка Android версии 13 и выше;
- Адаптация интерфейса для экранов с разрешением **от 720x1280 до 1440x2560 пикселей;**
- Корректная работа на устройствах с объемом оперативной памяти от 2 ГБ;

Тестируемость:

- Покрытие unit—тестами не менее 50% критического функционала
- Поддержка не менее 3 различных тестовых конфигураций устройств;
- Возможность тестирования в эмуляторах с разными характеристиками;
- **Наличие инструментов для нагрузочного тестирования API;**

Масштабируемость:

- Возможность одновременной работы не менее 1000 активных пользователей;
- Поддержка увеличения количества групп до 100 без потери производительности;
- Гибкая система кэширования часто запрашиваемых данных;

2 Анализ предметной области

2.1 Конкурентный анализ

Был проведён сравнительный анализ с двумя основными конкурентами: Cozi и FamilyWall. На рисунке (Рисунок 1) ниже отображены ключевые параметры, по которым происходило сравнение продуктов.

	Cozi	FamilyWall	ЗадачОК
Наличие ИИ	–	–	+
Доступность в РФ	–	–	+
Система поощрения	–	–	+
Статистика	+	+	+
Отслеживание покупок	+	+	–

Рисунок 1 — Ключевые параметры анализа конкурентов.

Cozi — популярное приложение на зарубежном рынке, ориентированное на совместное ведение семейных задач ([добавить ссылку на приложение или источники](#)). Оно позволяет всей семье использовать единый аккаунт, где можно синхронизировать календари, списки покупок, задачи и рецепты. Пользователи могут добавлять события в общий календарь, устанавливать напоминания, создавать детализированные списки покупок, вести списки дел, а также делиться ими между участниками семьи (Рисунок 2).

Однако Cozi не локализован под российский рынок, что делает его неудобным для большинства пользователей РФ, отсутствует интеграция с искусственным интеллектом, а также нет системы поощрений, [следовательно и конкретной мотивации для выполнения задач нет](#). Тем не менее, благодаря

простому интерфейсу и продуманной структуре, Cozi остаётся востребованным на своём рынке.

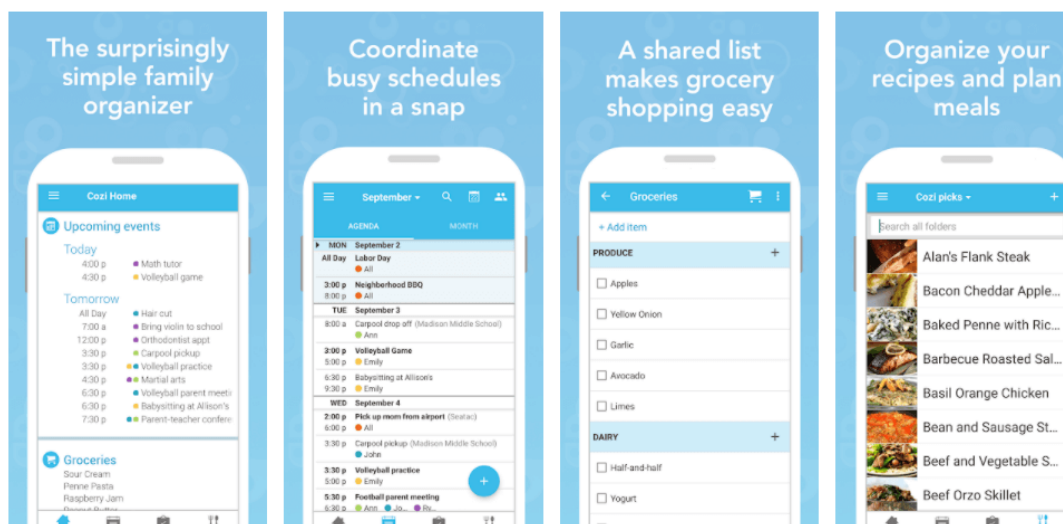


Рисунок 2 — Основные функции Cozi Family Organizer.

FamilyWall — это мобильное приложение и веб-сервис, предназначенные для управления семейной жизнью (добавить ссылку на приложение или источники). В одном интерфейсе объединены функции планирования, обмена сообщениями и отслеживания активности членов семьи (Рисунок 3).

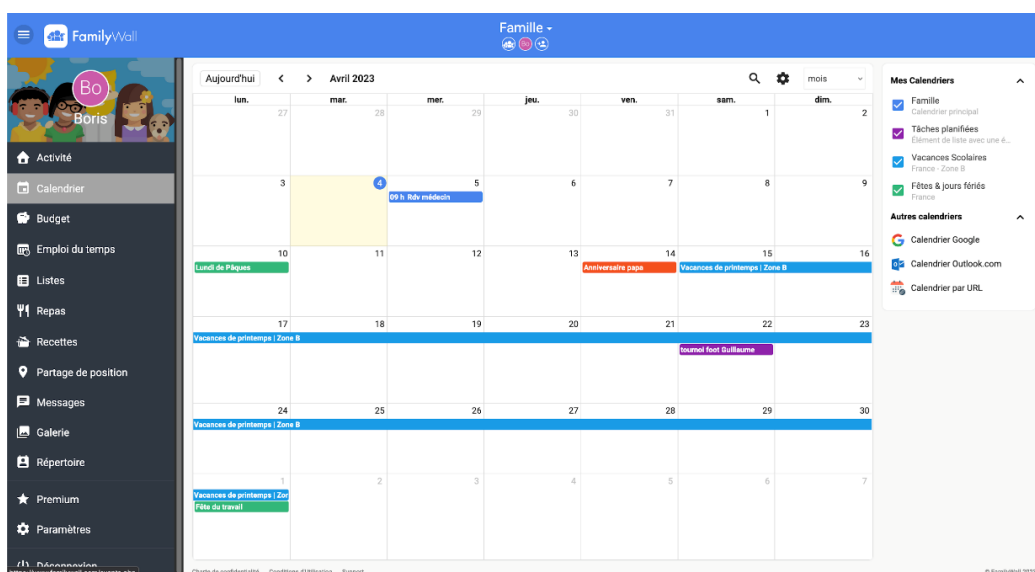


Рисунок 3 — Функции планирования FamilyWall

FamilyWall предлагает интерактивный календарь, в котором можно настраивать напоминания и делиться ими между участниками. Есть встроенный чат, общий семейный фотоальбом, раздел для рецептов, список покупок и дел.

Однако функциональность FamilyWall раскрывается в полной мере только в платной версии, а бесплатный доступ достаточно ограничен. Также сервис официально недоступен в России, не имеет русификации, и, как и Cozi, не использует ИИ и не предлагает систему поощрений за выполненные задачи.

2.2 Определение ЦА и потребностей

На основе анализа конкурентов можно сделать вывод, что существующие решения ориентированы, в первую очередь, на западный рынок и не учитывают специфические особенности российского пользователя.

Для определения реальных потребностей аудитории было проведено онлайн—исследование (результаты приведены в **приложении А**). В частности, на вопрос «Пользуетесь ли вы какими—либо приложениями для совместного ведения быта?» 86% опрошенных ответили отрицательно, что свидетельствует о **низкой насыщенности рынка подобными решениями или недостаточную заинтересованность пользователей, вызванную отсутствием действительно полезных, удобных и адаптированных под российский рынок сервисов**. При этом на вопрос «Хотели бы вы использовать приложение, которое бы отслеживало выполнение бытовых задач?» положительно ответили 65% участников. Это показывает, что спрос на подобный продукт уже сформирован. Таким образом, ЗадачОК выходит на рынок с уникальной комбинацией функций: интеграцией искусственного интеллекта, системой поощрений и полной русификацией интерфейса. Это позволяет приложению не только занять свободную нишу, но и сформировать новый стандарт ведения совместного быта в России.

3 Реализация

3.1 Frontend

Фронтенд—часть приложения реализована на кроссплатформенном фреймворке Flutter с использованием языка программирования Dart. Данный выбор обусловлен следующими преимуществами:

- Поддержка горячей перезагрузки (Hot Reload) для ускорения процесса разработки;
- Богатая экосистема готовых компонентов и библиотек;
- Высокая производительность за счет компиляции в нативный код.

3.1.1 Локальное состояние (виджетный уровень)

Для управления локальным состоянием интерфейсных компонентов применяется механизм `StatefulWidget` в сочетании с методом `setState()`. Данный подход обеспечивает реактивное обновление пользовательского интерфейса при изменении внутреннего состояния виджетов. Особое внимание уделяется корректной обработке жизненного цикла компонентов—перед каждым обновлением выполняется проверка флага `mounted`, что предотвращает попытки обновления уже удаленных из дерева виджетов. Этот уровень управления состоянием преимущественно используется для обработки пользовательских взаимодействий с формами ввода, управления видимостью вспомогательных элементов интерфейса и обработки временных состояний, связанных с конкретными экранами приложения. Реализация включает контроль за созданием и редактированием товаров и задач, переключением настроек интерфейса и динамическим отображением пользовательского контента.

3.1.2 Глобальное состояние (прикладной уровень)

Глобальное состояние приложения организовано с использованием библиотеки `Provider`, реализующей паттерн `InheritedWidget`, что обеспечивает централизованное управление данными и их согласованность во всех

компонентах системы. Ключевыми элементами архитектуры являются специализированные классы—провайдеры: `AuthProvider`, `SettingsProvider`, `TaskProvider`, `ShopProvider`, `GroupProvider`, каждый из которых инкапсулирует логику работы с определенной предметной областью и соответствующими API-эндпоинтами. `AuthProvider` отвечает за аутентификацию пользователей и управление сессией, автоматически добавляя JWT—токен в заголовки всех последующих запросов. `TaskProvider` осуществляет загрузку и обновление задач, реализуя кэширование данных для офлайн—работы и обеспечивая реакцию интерфейса на изменения. `ShopProvider` управляет виртуальным магазином, загружая ассортимент и обрабатывая покупки посредством, синхронизируя при этом баланс пользователя. `GroupProvider` координирует работу с группами пользователей, обрабатывая создание групп, приглашения участников и обновление состава. Все провайдеры используют единый экземпляр `ApiClient`, который инкапсулирует низкоуровневую логику HTTP—запросов, включая обработку ошибок, добавление обязательных заголовков и логирование операций. При получении данных от сервера провайдеры преобразуют сырые JSON—ответы в доменные модели, обновляют свое состояние и уведомляют подписчиков через механизм `notifyListeners()`, что вызывает автоматическое обновление всех зависимых виджетов (Рисунок 4).

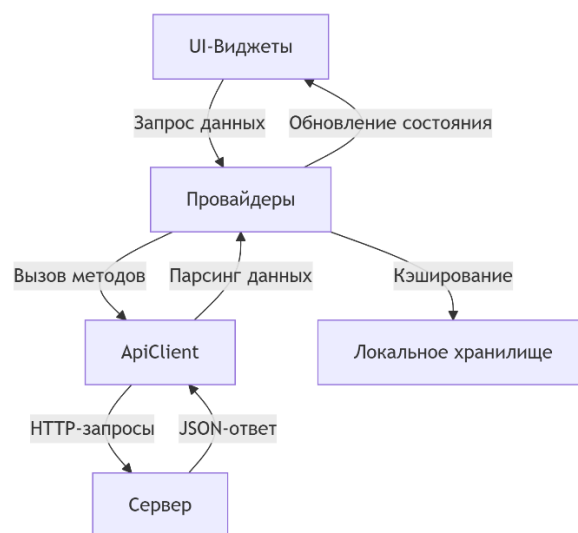


Рисунок 4 — Диаграмма управления состоянием и взаимодействия с сервером

Особенностью реализации является двухэтапная обработка данных: сначала выполняется валидация и нормализация на уровне ApiClient, затем бизнес—валидация в провайдерах перед обновлением состояния. Архитектура обеспечивает строгое разделение ответственности: виджеты отвечают только за отображение данных и обработку пользовательских жестов, провайдеры содержат бизнес—логику и состояние, а ApiClient инкапсулирует детали сетевого взаимодействия, что значительно упрощает поддержку и тестирование кода.

3.1.3 Персистентное состояние (хранение данных)

Для сохранения состояния между сессиями приложения используется механизм персистентного хранения данных на основе библиотеки `shared_preferences`. Реализация обеспечивает надежное сохранение настроек пользователя, включая токены доступа, настройки уведомлений и локальный кэш **часто используемых данных**. Особенностью данного подхода является автоматическое восстановление состояния при повторном запуске приложения, что создает эффект непрерывности пользовательского опыта. Все сохраняемые данные проходят процедуру **сериализации/десериализации**, обеспечивающую их целостность. Для защиты конфиденциальной информации применяются механизмы шифрования, предоставляемые платформой. Реализация учитывает ограничения мобильных устройств по объему хранимых данных и оптимизирована для работы с небольшими объемами структурированной информации.

3.1.4 Структура проекта

Архитектура клиентской части приложения организована по модульному принципу с четким разделением на слои, что соответствует современным подходам к разработке мобильных приложений. Основные компоненты структуры проекта включают следующие элементы:

3.1.4.1 Слои приложения

Проект разделен на три основных слоя: presentation (представление), business logic (бизнес—логика) и data (работа с данными). В слое представления находятся все виджеты и экраны приложения, которые отвечают исключительно за отображение информации и обработку пользовательских взаимодействий. Слой бизнес—логики содержит провайдеры которые инкапсулируют всю бизнес—логику приложения и управляют его состоянием. Слой работы с данными включает модели, отвечающие за взаимодействие с API и локальным хранилищем.

3.1.4.2 Организация экранов

Каждый экран приложения представляет собой самостоятельный модуль, содержащий весь необходимый код для его функционирования, не только UI—компоненты, но и обработчики событий, валидацию данных и логику взаимодействия с провайдерами. Для повторно используемых компонентов (кнопки, поля ввода, карточки задач) созданы отдельные виджеты, что обеспечивает единообразие интерфейса и упрощает поддержку кода.

3.1.4.3 Управления зависимостями

Все внешние зависимости и настройки темы вынесены в отдельные файлы конфигурации. Константы, используемые в интерфейсе (цвета, отступы, размеры шрифтов), определены в специализированных классах, что позволяет централизованно управлять визуальным стилем приложения. Для работы с API реализован модуль ApiClient, выступающий единой точкой входа для всех сетевых операций—этот класс инкапсулирует базовый URL, настройки и методы для выполнения запросов .

3.1.4.4 Про динамическую обработку интерфейса сюда добавить

3.1.4.5 Маршрутизация

Навигация между экранами организована через именованные маршруты с использованием Navigator 2.0, что обеспечивает предсказуемое поведение приложения и возможность глубоких ссылок. Все маршруты зарегистрированы в централизованном файле маршрутизации, что упрощает их модификацию и добавление новых экранов.

3.1.4.6 Тестирование и отладка

Архитектура проекта предусматривает возможность модульного тестирования всех компонентов. Ключевые бизнес—процессы вынесены в отдельные методы, что позволяет легко тестировать их изолированно. Для отладки сетевых запросов реализовано логирование всех входящих и исходящих данных.

3.2 Backend

3.2.1 Общая архитектура

Серверная часть приложения разработана по архитектурному шаблону «трёхуровневая архитектура» (Рисунок 5), где каждый уровень несёт строго определённую функциональную нагрузку. Основное разделение логики реализовано между уровнями контроллеров, сервисов и репозиторияв. Данный подход способствует модульности, повторному использованию кода, а также облегчает сопровождение и расширение программной системы.

Контроллеры отвечают за приём HTTP—запросов от клиента, их валидацию и передачу обработанных данных на уровень бизнес—логики. Сервисный слой инкапсулирует основную логику обработки информации и взаимодействует с уровнем репозиторияв, который, в свою очередь, обеспечивает доступ к базе данных посредством использования механизмов JPA и аннотаций Spring Data.

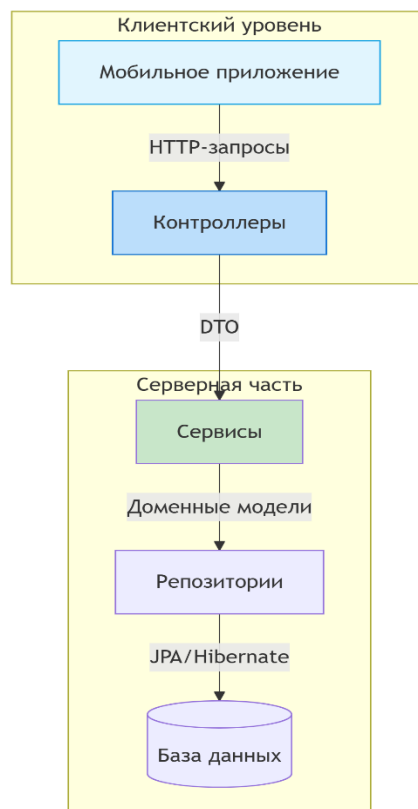


Рисунок 5 — Трёхуровневая архитектура

3.2.2 Контроллеры

Контроллеры реализуют API—интерфейс системы и выступают в роли посредника между клиентской частью и бизнес—логикой. Каждый контроллер соответствует определённой сущности предметной области, такой как пользователь (Customer), задача (Task), магазин (Shop), кошелёк (Wallet), лобби (Lobby) и т.д. В их обязанности входит получение параметров запроса, проверка корректности полученных данных, а также формирование и возврат ответа в формате JSON.

Обработка запросов осуществляется с использованием аннотаций Spring Framework, таких как `@RestController`, `@RequestMapping`, `@GetMapping`, `@PostMapping`, `@PutMapping` и `@DeleteMapping`. Каждый метод контроллера соответствует отдельной операции CRUD (создание, чтение, обновление, удаление), реализуемой над сущностью

3.2.3 Сервисный уровень

Сервисный уровень является центральным компонентом бизнес—логики приложения. В данном слое сосредоточена обработка входных данных, реализуются проверки условий, обеспечивается согласованность данных и осуществляется вызов методов, реализующих взаимодействие с хранилищем данных. Сервисный уровень получает данные от контроллеров и, при необходимости, передаёт их на уровень репозитория.

Реализация сервисов осуществляется посредством интерфейсов и их реализаций, помеченных аннотациями `@Service`. Подобная структура способствует расширяемости и удобству внедрения зависимости (Dependency Injection), что реализуется средствами Spring Container.

3.2.4 Репозитории и доступ к данным

Уровень репозитория предоставляет абстракцию над базой данных и реализует механизмы хранения, поиска и удаления информации. Использование Spring Data JPA позволяет упростить взаимодействие с базой данных за счёт автоматической генерации SQL—запросов на основе имён

методов. Каждой сущности в проекте соответствует отдельный интерфейс—репозиторий, расширяющий интерфейс `JpaRepository`, что даёт доступ ко множеству базовых операций.

Дополнительно в некоторых случаях определяются собственные запросы, реализуемые с помощью аннотаций `@Query`, или используются механизмы `Criteria API` для более сложных условий фильтрации и агрегации данных.

3.2.5 Обработка ошибок и безопасность

Особое внимание при проектировании backend— части уделено обработке ошибок. Для перехвата исключений, возникающих на различных уровнях приложения, реализован глобальный обработчик ошибок, использующий аннотацию `@ControllerAdvice`. Он позволяет централизованно управлять кодами HTTP—ответов и сообщениями об ошибках.

Кроме того, реализована система авторизации и аутентификации с использованием JWT (JSON Web Token). Каждый пользователь после успешного входа получает токен доступа, который должен быть прикреплен к каждому последующему запросу для подтверждения подлинности.

3.3 БД и СУБД

3.3.1 Описание базы данных AC ZadachOk

База данных автоматизированной системы `ZadachOk` реализована на основе реляционной модели данных (Рисунок 6). В качестве основной системы управления базами данных выбрана PostgreSQL версии 17, что обусловлено требованиями к надежности, производительности и масштабируемости системы.

Архитектура базы данных включает комплекс взаимосвязанных сущностей, отражающих предметную область системы распределения семейных обязанностей. В системе `Zadachok` пользователи создают задачи, которые могут иметь три состояния: 0—не выполнена, 1—выполнена (ожидает подтверждения админа), 2—подтверждена (начислены баллы). При

подтверждении задачи баллы поступают в кошелек пользователя, который привязан к конкретной группе.

Группы объединяют пользователей через `customer_id` и задачи через `task_id`, а также связаны с магазином. В магазине доступны товары, где `product_state` (`true/false`) показывает куплен товар или нет, а `customer_id` указывает на владельца. Пользователи тратят баллы из кошелька на покупку товаров в магазине своей группы.

Кошелек всегда привязан к конкретному пользователю и группе, что позволяет отслеживать баланс каждого участника в рамках разных групп. При подтверждении задачи баллы автоматически добавляются в кошелек исполнителя, а при покупке товара—списываются.

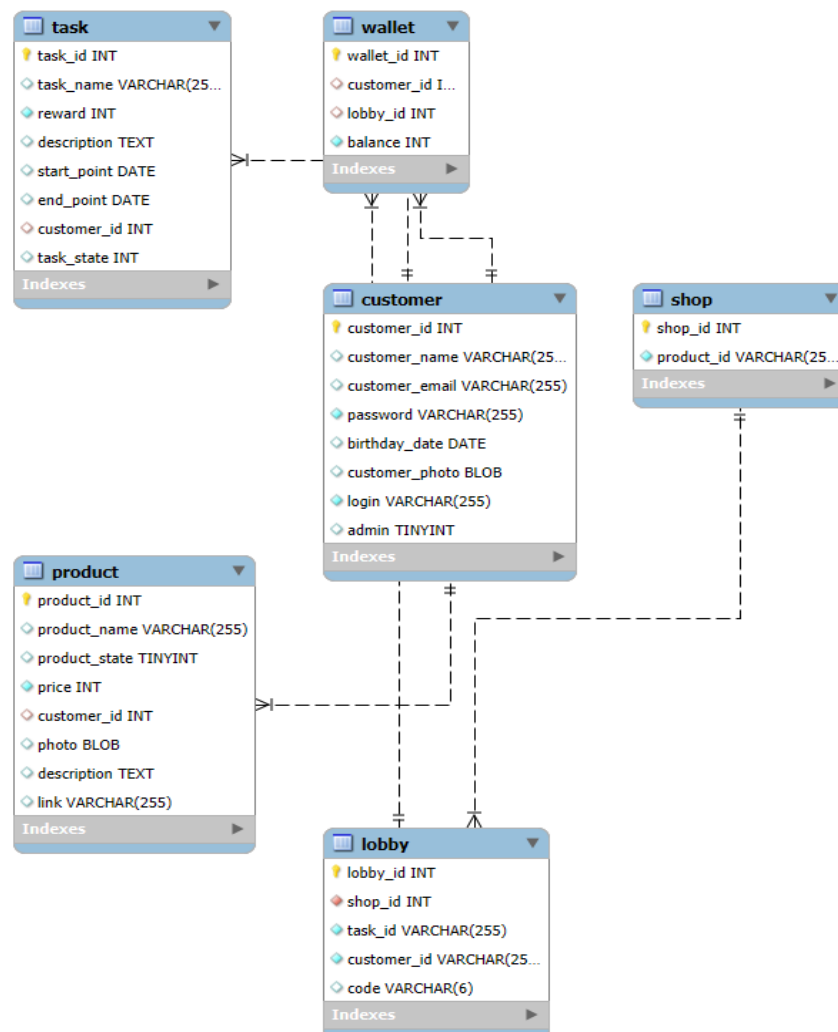


Рисунок 6 — ER—диаграмма

3.3.2 Безопасность, производительность и резервирование

Система ограничений базы данных гарантирует сохранение целостности при всех операциях. Реализованы каскадные обновления и удаления для связанных данных, проверки значений полей и уникальности ключевых атрибутов. Особое внимание уделено безопасности: пароли хранятся в хешированном виде, персональные данные шифруются, а все соединения используют защищенные протоколы. Разграничение доступа реализовано через комбинацию ролевой модели и ограничений на уровне SQL—запросов.

Для обеспечения высокой производительности в базе данных созданы составные индексы на часто используемых полях, оптимизированы наиболее ресурсоемкие запросы. Архитектура предусматривает возможность шардинга данных пользователей при росте нагрузки. Реализовано кэширование результатов сложных запросов, таких как статистика выполнения задач. Эти решения позволяют системе сохранять отзывчивость при увеличении количества пользователей до 10 000 активных сессий.

Система миграций на основе Flyway обеспечивает контроль версий схемы базы данных и возможность отката изменений. Ежедневные полные бэкапы и почасовые инкрементальные копии гарантируют восстановление данных при любых сбоях. Процедуры резервного копирования включают проверку целостности сохраненных данных и автоматическое оповещение администратора в случае обнаружения проблем. Все изменения в структуре базы данных фиксируются в журнале изменений с указанием автора и цели модификации.

3.4 Интеграция искусственного интеллекта (переименовать)

3.5 Развёртывание и запуск (переименовать)

3.5.1 Процесс развертывания

Развертывание системы осуществлялось поэтапно с последовательным решением возникающих технических сложностей. Первоначальная попытка контейнеризации при помощи Docker выявила проблемы сетевой конфигурации—порты API оставались недоступны для внешних подключений из—за ограничений хостинг—провайдера, а внутренняя маршрутизация между контейнерами функционировала некорректно вследствие ошибочной настройки сети Docker. Данные обстоятельства потребовали временного отказа от контейнеризации в пользу ручного развертывания компонентов системы.

Ручная настройка серверного окружения потребовала выполнения ряда критически важных операций. Была проведена детальная настройка systemd для управления Java—процессом API с оптимизацией параметров JVM в соответствии с доступными серверными ресурсами. Конфигурация Nginx в качестве reverse proxy потребовала глубокой проработки механизмов SSL/TLS и особенностей проксирования web—запросов. В процессе эксплуатации был зафиксирован инцидент кибербезопасности, когда сервер оказался скомпрометирован через уязвимость в системе защиты. Данный случай подтвердил необходимость строгого соблюдения политик безопасности, включая своевременное обновление ПО, постоянный мониторинг активности и жесткий контроль прав доступа.

В результате проведенных работ была достигнута стабильная работа системы с соблюдением всех требований к производительности и безопасности. Полученный опыт позволил сформировать оптимальную конфигурацию инфраструктуры и выработать эффективные процедуры поддержки и мониторинга.

3.5.2 Финальная архитектура и реализация

На основе полученного опыта была разработана оптимизированная архитектура решения с использованием Docker (Рисунок 7). Реализация включала следующие ключевые аспекты:

Контейнер PostgreSQL был развернут с персистентным хранилищем данных через volumes и настроен с жесткими ограничениями доступа. Для Java—API применен принцип минимальных привилегий, все чувствительные данные вынесены в Docker Secrets. Конфигурация Nginx усилена поддержкой TLS 1.3, HTTP/2 и строгими security—политиками, включая HSTS и современные шифрсы.

Обеспечена сетевая изоляция контейнеров через dedicated Docker—сеть с явно заданными правилами взаимодействия. Внедрена комплексная система мониторинга, объединяющая централизованное логирование и сбор метрик для оперативного выявления аномалий.

Процесс CI/CD полностью автоматизирован: каждый коммит проходит через pipeline с выполнением unit— и интеграционных тестов, анализом безопасности образов (trivy) и автоматическим развертыванием на staging—окружении. Только после успешного прохождения всех проверок происходит деплой в production.

Финальная архитектура обеспечивает:

- Отказоустойчивость за счет изолированных контейнеров и персистентного хранения;
- Безопасность через принцип минимальных привилегий и автоматизированные проверки;
- Масштабируемость благодаря контейнеризации и разделению сервисов;
- Надежность обновлений через многоступенчатый CI/CD—процесс.

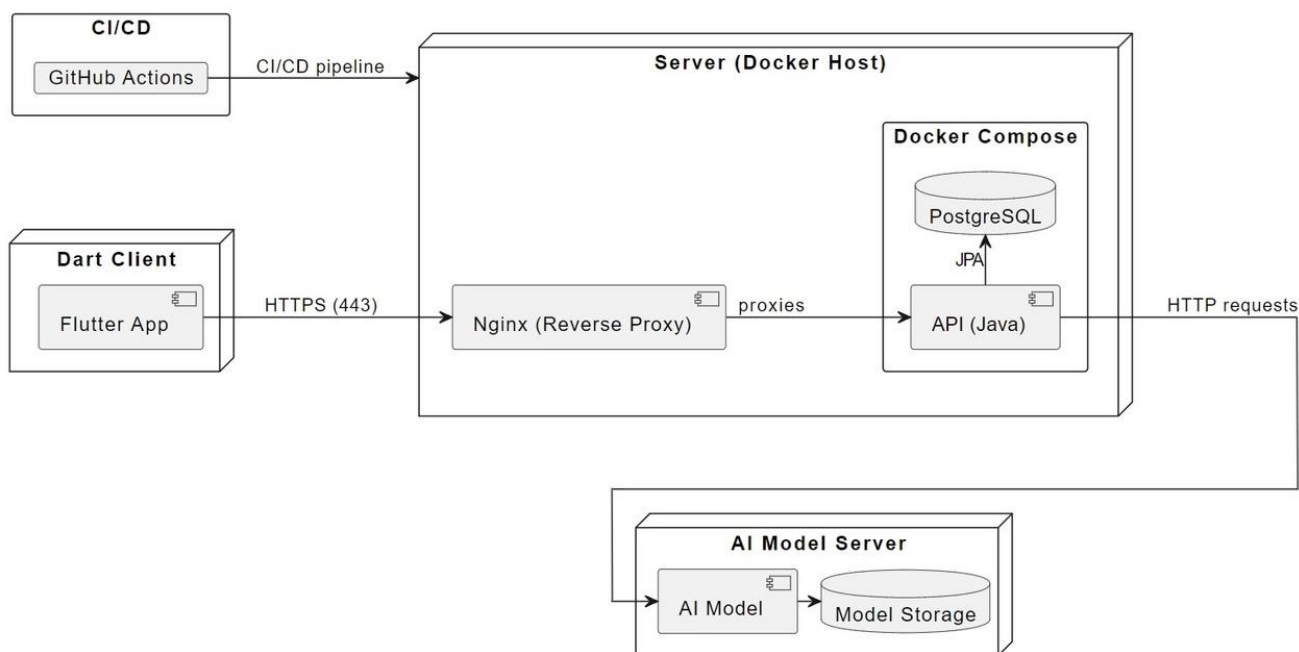


Рисунок 7 — Схема развертывания

Данное решение не только соответствует текущим требованиям, но и закладывает основу для будущего развития системы, включая возможности горизонтального масштабирования и blue—green деплоев.

3.6 CI/CD и тестирование

3.7 Функциональные возможности

3.7.1 Роли пользователей в системе

В системе реализована многоуровневая модель доступа с четырьмя ключевыми ролями пользователей, обеспечивающая постепенное расширение функциональных возможностей по мере вовлечения пользователя в работу приложения (Рисунок).

Базовой ролью является неавторизованный пользователь, который имеет минимальный набор функций:

- Просмотр календаря в ограниченном режиме без возможности взаимодействия с задачами;
- Доступ только к базовым настройкам интерфейса без редактирования персональных данных;
- возможность пройти процедуру регистрации для получения расширенного доступа.

После успешной регистрации пользователь переходит в статус авторизованного, но без привязки к конкретной группе—на этом этапе становятся доступны функции полноценной работы с профилем, включая:

- Редактирование личных данных
- Возможность создания собственной группы с автоматическим получением прав администратора или ожидания приглашения в существующее лобби.

Особенностью данного промежуточного статуса является невозможность взаимодействия с задачами и виртуальным магазином до момента вступления в группу. После получения приглашения и вступления в группу пользователь приобретает статус участника с расширенным функционалом:

- Появляется возможность принимать, создавать и отмечать выполнение задач;
- Получать баллы за завершённые задания;
- Тратить накопленные баллы в магазине группы;
- Просмотр персональной статистики.

Максимальными правами обладает администратор группы, который помимо стандартного функционала участника получает инструменты управления:

- Создание и редактирование задач с назначением вознаграждений;
- Полный контроль над ассортиментом виртуального магазина;
- Возможность приглашать новых участников (по коду группы);
- Также доступ ко всем настройкам группы.

Такая иерархическая система ролей обеспечивает логичное и безопасное распределение прав, позволяя каждому пользователю получить доступ только к тем функциям, которые соответствуют его текущему уровню вовлеченности в систему.

3.7.2 Функциональные возможности

(здесь описать каждый пользовательский путь)

3.7.3 Режимы функционирования системы

3.7.3.1 Онлайн-режим

Обеспечивает полный функционал приложения при наличии интернет-соединения. В данном режиме пользователи могут создавать и редактировать задачи, назначать вознаграждения, отмечать выполнение заданий, совершать покупки в виртуальном магазине, а также просматривать актуальную статистику. Все операции синхронизируются с сервером в реальном времени, включая: обновление статусов задач, начисление баллов за выполненные

задания, изменения в составе групп и ассортименте магазина. Администраторы групп получают возможность управлять настройками лобби, добавлять новых участников и редактировать параметры вознаграждений. Особенностью онлайн-режима является работа системы push-уведомлений, текст для которых генерируются искусственным интеллектом на основе анализа текущих задач и активности пользователей.

3.7.3.2 Оффлайн-режим

Активируется автоматически при отсутствии интернет-соединения и предоставляет ограниченный функционал. Пользователи могут просматривать ранее загруженные данные: список задач с их текущими статусами, информацию о группе, доступные товары в магазине и личный баланс баллов. Доступны базовые операции: создание новых задач и редактирование существующих (изменения сохраняются локально), просмотр календаря с запланированными заданиями, работа с настройками профиля. Важно отметить, что в оффлайн-режиме недоступны: синхронизация данных между устройствами, начисление баллов за выполненные задания, покупки в виртуальном магазине, а также получение уведомлений, генерируемых ИИ. Все изменения, сделанные в оффлайн-режиме, автоматически синхронизируются с сервером при восстановлении интернет-соединения, обеспечивая целостность данных системы.

Переход между режимами осуществляется автоматически и незаметно для пользователя, что обеспечивает непрерывность работы с приложением независимо от качества интернет-соединения. Система сохраняет все внесённые изменения и гарантирует их актуальность после восстановления связи с сервером.

Заключение

В ходе выполнения курсовой работы было разработано и успешно протестировано мобильное приложение ZadachOk для распределения семейных обязанностей, соответствующее всем поставленным техническим требованиям. Реализованная система предоставляет комплексное решение для организации совместного выполнения задач в семейном кругу с элементами геймификации.

Основные достижения проекта:

- Разработана полнофункциональная мобильная система на базе Flutter с серверной частью на Spring Boot;
- Реализована четырёхуровневая система ролей пользователей (неавторизованный, авторизованный, участник, администратор) с дифференцированным доступом к функциям;
- Создана система мотивации через виртуальную валюту и магазин вознаграждений;
- Обеспечена работа в двух режимах (онлайн/оффлайн) с автоматической синхронизацией данных;
- Развернута отказоустойчивая инфраструктура на базе Docker с автоматизированным CI/CD pipeline;
- **Про ИИ.**

Тестирование системы включало:

- Модульное тестирование критических компонентов (покрытие >50%);
- Интеграционное тестирование API endpoints;
- Нагрузочное тестирование до 1000 одновременных пользователей;
- Юзабилити-тестирование интерфейса на 3 типах устройств;

Ключевые функциональные возможности:

- Создание и распределение задач между членами семьи;
- Система баллов за выполнение заданий;
- Виртуальный магазин для обмена баллов на вознаграждения;
- Интеллектуальные уведомления [функция в разработке];
- Личная статистика (заданий/день);
- Гибкие настройки групп и прав доступа.

Перспективы развития:

.....что-нибудь тут

Разработанное приложение успешно решает поставленные задачи организации семейного быта и мотивации домочадцев, демонстрируя стабильную работу на всех этапах тестирования. Полученные результаты подтверждают соответствие системы заявленным требованиям и возможность дальнейшего масштабирования функционала.

Список использованных источников

- Официальная документация Flutter SDK;
- Официальная документация Dart;
- Официальная документация Java;
- Официальная документация Lombok;
- Официальная документация Spring Boot;
- Официальная документация PostgreSQL;
- ГОСТ 34.201-20. Информационная технология. Комплекс стандартов на автоматизированные системы. Виды, комплектность и обозначение документов при создании автоматизированной системы;
- ГОСТ 2.105-19. ЕСКД. Общие требования к текстовым документам.
(дополнить/убрать и отформатировать)

оформить всё по ГОСТ

ПРИЛОЖЕНИЕ А

Сюда по-хорошему диаграммы, брендбук и айконкит.